

MPC Smart-Meter Testbed Codebase Handoff and Improvement Plan

Current State

The codebase currently implements and validates three related systems:

1. A C-based additive-secret-sharing MPC benchmark.
2. A CSV-backed smart-meter emulator using the same MPC runner.
3. A high-N central secure aggregation stress path for 1000 simulated meters.

The current machines are:

Machine	Role
PC 5	Development and orchestration machine
Pi 5	Raspberry Pi 5, higher-capacity host, Tailscale IP <code>100.101.20.65</code>
Pi 2	Raspberry Pi 2, lower-capacity host, Tailscale IP <code>100.85.83.7</code>

Current repo paths:

Machine	Path
PC 5	<code>~/Desktop/Work/Hanze/work/mpc-smartgrid</code>
Pi 5	<code>~/mpc-smartgrid</code>
Pi 2	<code>~/mpc-smartgrid</code>

Important deployment rule:

```
Never sync build/ from PC 5 to either Pi.  
PC 5 should sync source only.  
Each Pi must rebuild its own binaries locally.
```

Use:

```
rsync -av --delete --exclude build/ --exclude logs/ --exclude  
benchmark_results/  
~/Desktop/Work/Hanze/work/mpc-smartgrid/  
solomon@100.101.20.65:~/mpc-smartgrid/
```

```
rsync -av --delete --exclude build/ --exclude logs/ --exclude  
benchmark_results/  
~/Desktop/Work/Hanze/work/mpc-smartgrid/  
pi@100.85.83.7:~/mpc-smartgrid/
```

Then build on each Pi:

```
cd ~/mpc-smartgrid  
make clean  
make
```

Validated Results So Far

The following experiments have already passed.

1. Two-party MPC benchmark

```
Pi 5 secret = 300  
Pi 2 secret = 700  
Expected aggregate = 1000  
Observed aggregate = 1000  
Status = PASS
```

2. Six-party MPC benchmark

```
Nodes 0,1,2 on Pi 5  
Nodes 3,4,5 on Pi 2  
Inputs = 10,20,30,40,50,60  
Expected aggregate = 210  
Observed aggregate = 210  
Status = PASS
```

3. Smart-meter emulator

```
Pi 5 meter input = 312  
Pi 2 meter input = 500  
Expected aggregate = 812  
Observed aggregate = 812  
Status = PASS
```

Repeated smart-meter rounds also passed:

```
312 + 500 = 812
318 + 520 = 838
305 + 510 = 815
```

4. Automated constrained two-party benchmark

The automated 12-case benchmark matrix passed under baseline, CPU-constrained, memory-constrained, and combined CPU/memory-constrained Pi 5 runs.

Strongest constrained example:

```
Pi 5: CPUQuota=25%, MemoryMax=16M
Pi 2: normal peer
Expected aggregate = 812
Observed aggregate = 812
Status = PASS
```

5. 1000-meter secure aggregation stress test

The high-N secure aggregation path passed two full 1000-meter runs.

Configuration:

```
N = 1000 simulated meters
Pi 5 meters = 900
Pi 2 meters = 100
Per-meter memory cap = 16384 KB
Per-meter FD cap = 64
Per-meter CPU time cap = 10 sec
Pi 5 group cap = CPUQuota=300%, MemoryMax=1024M
Pi 2 group cap = CPUQuota=100%, MemoryMax=256M
Expected aggregate = 536600
```

Successful run 1:

```
Observed aggregate = 536600
Received submissions = 1000/1000
Wall time = 10585.106 ms
Aggregator RSS = 1056 KB
Status = PASS
```

Successful run 2:

```
Observed aggregate = 536600
Received submissions = 1000/1000
Wall time = 10762.754 ms
Aggregator RSS = 1072 KB
Status = PASS
```

One later 1000-meter run failed due to orchestration or timeout behavior, not a proven arithmetic or aggregation correctness bug. This needs reliability hardening.

Codebase Map

Important files:

```
src/mpc_field.c
src/mpc_share.c
src/mpc_comm_tcp.c
include/mpc_runner.h
src/mpc_runner.c
test/bench_mpc.c
apps/meter_node.c
apps/secure_agg_server.c
apps/secure_agg_meter.c
scripts/bench_worker.sh
scripts/run_suite_pc5.sh
scripts/run_scale_suite_pc5.sh
scripts/secure_agg_launch_range.sh
scripts/run_final_1000_secure_agg_pc5.sh
configs/cluster_test2.json
configs/cluster.json
```

Build targets:

```
build/test_field
build/test_share
build/test_uc1
build/bench_mpc
build/meter_node
build/secure_agg_server
build/secure_agg_meter
```

Main Technical Distinction

There are two different protocol paths. Do not mix them up.

All-party MPC path

Used by:

```
bench_mpc
meter_node
mpc_runner
```

Properties:

```
All parties exchange shares.
All parties reconstruct the aggregate.
Communication is roughly  $O(N^2)$ .
Validated for  $N=2$  and  $N=6$ .
```

This is the original MPC benchmark path.

Central secure aggregation path

Used by:

```
secure_agg_server
secure_agg_meter
```

Properties:

```
Each meter sends one masked value to central aggregator.
Pairwise masks cancel in the final sum.
Aggregator reconstructs only the total aggregate.
Communication is roughly  $O(N)$ .
Validated for  $N=1000$  in two successful runs.
```

This is not 1000-party all-party MPC. It is a central secure aggregation stress test.

Priority 1: Clean Up and Harden the Code

1. Move shared constants into headers

Right now constants may be duplicated across files.

Create something like:

```
include/mpc_limits.h
```

Put:

```
#ifndef MPC_LIMITS_H
#define MPC_LIMITS_H

#define MPC_MAX_PARTIES 256
#define MPC_CONFIG_BUF_SIZE 65536
#define MPC_DEFAULT_RECV_TIMEOUT 30

#endif
```

Then replace local constants such as:

```
#define RUNNER_MAX_PARTIES ...
#define BENCH_MAX_PARTIES ...
```

with the shared constant.

Reason: if one file says 64 and another says 256, scaling tests will fail unpredictably.

2. Replace the hand-written JSON parser

The current config parser is lightweight and fragile. It is acceptable for early tests, but it is not robust.

Options:

1. Keep current parser but add strict validation.
2. Use a tiny C JSON parser such as `jsmn`.
3. Generate simpler config files that avoid complex JSON parsing.

Recommended short-term fix: keep the parser but validate:

```
all ids from 0..N-1 exist
no duplicate ids
host is non-empty
port is positive
N <= MPC_MAX_PARTIES
config file size fits buffer
```

3. Add structured error codes

Right now many failures return `1`. Improve this.

Use enum-style error codes:

```
typedef enum {  
    MPC_OK = 0,  
    MPC_ERR_CONFIG = 1,  
    MPC_ERR_CONNECT = 2,  
    MPC_ERR_SEND = 3,  
    MPC_ERR_RECV = 4,  
    MPC_ERR_TIMEOUT = 5,  
    MPC_ERR_AGGREGATE = 6  
} mpc_status_t;
```

Reason: benchmark scripts can classify failures automatically.

4. Make logs machine-readable

Current logs are human-readable. Keep them, but also add CSV or JSON lines.

For each run, write:

```
test_id,node_id,n_parties,input,aggregate,protocol_ms,max_rss_kb,status,error_code,error_message
```

For secure aggregation server, write:

```
test_id,N,expected,observed,received,duplicates,invalid,wall_ms,max_rss_kb,status
```

5. Fix high-concurrency orchestration reliability

The 1000-meter secure aggregation path passed twice, then failed once. The likely issue is orchestration, cleanup, or timeout behavior.

Improve:

```
kill stale server and meter processes before every run  
verify port 7000 is free before server launch  
wait until aggregator is actually listening before launching clients  
wait for aggregator process exit, not just sleep  
capture server exit code
```

```
capture launcher exit code
capture per-Pi meter failure count
```

Add an explicit server readiness check:

```
until ssh solomon@100.101.20.65 'ss -ltn | grep -q ":7000"'; do
    sleep 0.2
done
```

6. Add retry logic for client launch waves

Launching 1000 Linux processes at once can cause scheduling and connection bursts.

Improve `secure_agg_launch_range.sh`:

```
launch clients in batches of 50 or 100
sleep 100-300 ms between batches
record batch start/end timestamps
```

This will make the run more stable and easier to analyze.

7. Add dropout-aware server timeout

The current server waits for N submissions until timeout kills it. Add a timeout option inside the server:

```
./secure_agg_server --n 1000 --port 7000 --expected 536600 --timeout-ms 300000
```

Then the server itself can print:

```
received_count
missing_count
missing_ids
status
```

This is better than external `timeout`.

8. Track missing meter IDs

The server currently tracks seen IDs. At timeout or completion, print missing IDs.

Add:


```
MISSING_IDS = 17,93,402,...
```

For large failures, write missing IDs to a file:

```
logs/final_1000/missing_ids.txt
```

This makes debugging much faster.

9. Add authentication fields later

Current secure aggregation packets are minimal:

```
magic  
id  
masked_value
```

For a stronger prototype, add:

```
test_id  
nonce  
protocol_version  
packet_type
```

Do not add real crypto unless the design is ready, but at least prevent accidental cross-run contamination.

10. Improve secure aggregation security model

Current mask derivation uses a shared `mask_key` known to the test harness. This is acceptable for stress testing but not production secure aggregation.

Future improvement:

```
each pair of meters derives pairwise masks from pairwise keys  
aggregator never knows pairwise masks  
keys are established through ECDH or pre-shared pair keys  
dropout recovery is added
```

Do not overclaim privacy until this is done.

Priority 2: Add Proper Test Scripts

1. Smoke test script

Create:

```
scripts/test_smoke_pc5.sh
```

Purpose:

```
sync source
build both Pis
run make test on both Pis
run 2-party bench_mpc
run 2-party meter_node
```

Expected outputs:

```
bench_mpc aggregate = 1000
meter_node aggregate = 812
```

This should be the first script run after every code change.

2. Two-party constraint suite

Current script already exists:

```
scripts/run_suite_pc5.sh
```

Improve it to support a CLI:

```
./scripts/run_suite_pc5.sh --trials 5 --out benchmark_results/constraint_run
```

It should test:

```
baseline
CPUQuota=75%
CPUQuota=50%
CPUQuota=25%
MemoryMax=64M
```

```
MemoryMax=32M
MemoryMax=16M
CPUQuota=50% + MemoryMax=32M
CPUQuota=25% + MemoryMax=32M
CPUQuota=25% + MemoryMax=16M
```

Output:

```
final_matrix.csv
raw logs per test case
summary.txt
```

3. Repeated smart-meter rounds

Create or improve:

```
scripts/test_repeated_meter_rounds_pc5.sh
```

Cases:

```
round 0: 312 + 500 = 812
round 1: 318 + 520 = 838
round 2: 305 + 510 = 815
```

Run each round 10 times.

Output:

```
round,trial,expected,observed_pi5,observed_pi2,pi5_ms,pi2_ms,status
```

4. Six-party benchmark regression

Create:

```
scripts/test_6party_pc5.sh
```

Layout:

```
nodes 0,1,2 on Pi 5
nodes 3,4,5 on Pi 2
```

```
values 10,20,30,40,50,60
expected 210
```

This should clean ports, start all nodes through tmux, collect logs, and verify all six report 210.

5. All-party N-scaling script

Current script:

```
scripts/run_scale_suite_pc5.sh
```

Improve it to support:

```
./scripts/run_scale_suite_pc5.sh --n-values 2,4,6,8,10,12,16 --trials 3
```

Metrics:

```
N
trial
expected
completed_nodes
all_match
mean_protocol_ms
max_protocol_ms
status
failure_reason
```

This evaluates the original all-party MPC path. It should not be confused with the secure aggregation path.

6. Extreme secure aggregation script

Current script:

```
scripts/run_final_1000_secure_agg_pc5.sh
```

Improve it into a general script:

```
scripts/run_secure_agg_suite_pc5.sh
```

CLI:

```
./scripts/run_secure_agg_suite_pc5.sh
--n 1000
--trials 10
--pi2-min 100
--pi2-max 300
--meter-mem-kb 16384
--pi5-group-cpu 300%
--pi5-group-mem 1024M
--pi2-group-cpu 100%
--pi2-group-mem 256M
```

Output:

```
secure_agg_matrix.csv
per-run aggregator logs
per-run meter logs
missing_ids.txt for failed runs
```

7. Secure aggregation load sweep

Create:

```
scripts/run_secure_agg_load_sweep_pc5.sh
```

Test:

```
N = 250
N = 500
N = 750
N = 1000
N = 1250
N = 1500
```

Each with 3 trials.

Stop after first N with repeated failure, or continue and mark failures.

8. Pi 2 placement sweep

Create:

```
scripts/run_pi2_placement_sweep_pc5.sh
```

Purpose: find how much Pi 2 can safely carry.

Fixed:

```
N = 1000
```

Vary:

```
Pi 2 meters = 50  
Pi 2 meters = 100  
Pi 2 meters = 150  
Pi 2 meters = 200  
Pi 2 meters = 300
```

Pi 5 gets the rest.

This answers:

```
At what Pi 2 load does repeated-run reliability collapse?
```

9. Memory sweep

Create:

```
scripts/run_memory_sweep_pc5.sh
```

For secure aggregation:

```
N = 1000  
Pi 5 = 900  
Pi 2 = 100  
per-meter memory = 32768 KB  
per-meter memory = 16384 KB  
per-meter memory = 8192 KB  
per-meter memory = 4096 KB
```

Run 3 trials each.

This determines the minimum viable per-meter memory cap.

10. Network impairment script

Create:

```
scripts/run_netem_tests_pc5.sh
```

It should apply `tc netem` on Pi 5 or both Pis:

```
baseline
delay 100ms
delay 100ms jitter 20ms
delay 100ms loss 1%
delay 100ms loss 3%
```

Run both:

```
2-party meter_node
1000-meter secure aggregation
```

Always remove netem after the test:

```
sudo tc qdisc del dev tailscale0 root
```

Add cleanup traps so netem is removed even if the script fails.

11. Long-running stability test

Create:

```
scripts/run_long_stability_pc5.sh
```

Test:

```
2-party meter_node every 5 seconds for 1 hour
secure aggregation N=1000 every 2 minutes for 1 hour
```

Record:

```
trial number
timestamp
status
wall time
received count
observed aggregate
failure reason
```

This is necessary for reliability claims.

Priority 3: Improve C Code Correctness and Maintainability

1. Replace `atoi` and `atol`

Use `strtol` / `strtoul` with validation.

Bad:

```
int n = atoi(argv[++i]);
```

Better:

```
char *end = NULL;
long v = strtol(argv[++i], &end, 10);
if (*end != '\0' || v < 0 || v > INT_MAX) {
    error;
}
```

This prevents silent parsing bugs.

2. Add bounds checks everywhere

Check:

```
id >= 0
id < n
n <= max supported clients
value < MPC_PRIME
port in valid range
host string length
file paths not null
```


3. Add versioned packet formats

Secure aggregation packet should include:

```
typedef struct {  
    uint32_t magic;  
    uint16_t version;  
    uint16_t type;  
    uint32_t test_id_hash;  
    uint32_t id;  
    uint32_t masked_value;  
} packet_t;
```

This prevents old clients from accidentally talking to a new server.

4. Add server-side timeout

The server should not rely only on shell `timeout`.

Add CLI:

```
--timeout-ms 300000
```

Use `poll`, `select`, or socket timeout.

5. Add missing ID reporting

At the end of server run:

```
received_count  
missing_count  
missing_ids
```

This is essential for stress tests.

6. Add retry/backoff to meter clients

Current clients retry connection, but all clients may retry in lockstep.

Add jitter:

```
sleep_ms = base + random_jitter
```

This reduces thundering herd behavior.

7. Avoid launching 1000 processes at exactly once

Use batch launch:

```
batch_size = 50  
batch_gap_ms = 200
```

This improves stability without weakening the test.

8. Add build modes

Makefile should support:

```
make debug  
make asan  
make release
```

Recommended flags:

```
debug: -O0 -g -Wall -Wextra  
asan: -O1 -g -fsanitize=address,undefined  
release: -O2
```

ASAN may not be practical on Pi 2, but it is useful on PC 5.

9. Add `make format`

Use `clang-format` if available.

10. Add `make lint`

Use:

```
cppcheck --enable=all --inconclusive --std=c99 src apps test
```

Priority 4: Test Matrix They Should Implement

The final testing plan should include these layers.

Layer A: Local unit tests

Run on PC 5 and both Pis:

```
field arithmetic
share split/reconstruct
mask cancellation test
packet encoding/decoding test
value generation test
expected aggregate test
```

Layer B: Local integration tests

Run on one machine:

```
secure_agg_server + 10 local clients
secure_agg_server + 100 local clients
```

This isolates network problems from protocol problems.

Layer C: Distributed smoke tests

Run on Pi 5 and Pi 2:

```
2-party bench_mpc
6-party bench_mpc
2-party meter_node
```

Layer D: Constraint tests

Run:

```
CPU 75%, 50%, 25%
Memory 64M, 32M, 16M
Combined CPU/memory constraints
```

Layer E: Secure aggregation scale tests

Run:

```
N = 100, 250, 500, 750, 1000, 1250, 1500
```

For each:

```
3 trials minimum
record pass rate
record wall time
record received count
record aggregator RSS
record missing IDs
```

Layer F: Reliability tests

Run:

```
N=1000, 10 trials
N=1000, 30 trials if time allows
```

Report:

```
success rate
mean wall time
p95 wall time
failure categories
```

Layer G: Adversarial tests

Test:

```
duplicate meter submissions
invalid meter ID
missing meter submissions
bad masked value >= MPC_PRIME
wrong magic number
wrong test ID
server starts late
```

```
clients start before server  
port already occupied
```

Layer H: Network impairment

Test with `tc netem`:

```
delay  
jitter  
packet loss  
delay + loss
```

Layer I: Dropout tests

Simulate:

```
1% dropout  
5% dropout  
10% dropout
```

Current secure aggregation will fail correctness under dropout unless dropout recovery is added. That is expected and should be documented.

Priority 5: Reporting Improvements

Every script should output a machine-readable CSV with:

```
run_id  
timestamp  
test_name  
N  
pi5_meters  
pi2_meters  
expected  
observed  
received  
status  
reason  
wall_ms  
aggregator_rss_kb  
pi5_group_cpu  
pi5_group_mem  
pi2_group_cpu
```

```
pi2_group_mem
per_meter_mem_kb
per_meter_fd_cap
per_meter_cpu_seconds
```

For failed runs, include:

```
missing_count
missing_ids_file
failed_meter_logs_count
server_exit_code
pi5_launcher_exit_code
pi2_launcher_exit_code
```

Do not rely on screenshots. Every claim should come from a CSV or log file.

Immediate Tasks for Friends

They should do the following in order.

Task 1: Freeze current working state

Create a branch:

```
git checkout -b stable-current-results
git add .
git commit -m "Stable MPC and secure aggregation testbed results"
```

Then create a new branch for improvements:

```
git checkout -b testbed-hardening
```

Task 2: Add smoke test script

Implement `scripts/test_smoke_pc5.sh`.

It should validate:

```
build on both Pis
make test on both Pis
```

```
2-party bench_mpc = 1000  
2-party meter_node = 812
```

Task 3: Harden 1000-meter script

Improve:

```
server readiness check  
proper wait for aggregator exit  
batch client launch  
missing ID reporting  
server timeout  
better failure logs
```

Task 4: Run 10 trials of N=1000

Use current settings:

```
Pi 5 = 900  
Pi 2 = 100  
per-meter memory = 16 MB
```

Report:

```
PASS count / 10  
mean wall_ms  
p95 wall_ms  
failure reasons
```

Task 5: Implement load sweep

Run:

```
N = 250, 500, 750, 1000, 1250, 1500
```

This gives the capacity curve.

Task 6: Implement Pi 2 placement sweep

Run:

```
N=1000  
Pi2=50,100,150,200,300
```

This identifies Pi 2's practical limit.

Task 7: Implement memory sweep

Run:

```
per-meter memory = 32 MB, 16 MB, 8 MB, 4 MB
```

This identifies minimum process memory.

Task 8: Implement network impairment tests

Run:

```
baseline  
100 ms delay  
100 ms delay + 1% loss  
100 ms delay + 3% loss
```

Task 9: Add dropout tests

Expected behavior today:

```
dropout causes incomplete aggregation
```

Future work:

```
dropout-resilient secure aggregation
```

Task 10: Prepare final report tables

Generate tables for:

```
all-party MPC correctness  
2-party constrained smart-meter benchmark  
1000-meter secure aggregation  
secure aggregation reliability
```


load sweep
memory sweep
Pi 2 placement sweep
network impairment

Final Warning

Do not claim:

1000-party all-party MPC works.
Production smart-grid security is solved.
STM32/MSP430 deployment is complete.
Dropout-resilient aggregation is implemented.

Correct claim:

The testbed validates small-N all-party MPC and demonstrates high-N central secure aggregation feasibility on two Raspberry Pis using constrained Linux processes.